# Lab 05 — Services, Selectors, Endpoints & DNS

**ID:** K8S-LAB-05 · **Category:** Networking · **Level:** Engineer · **Duration:** 60 min

**Focus:** Service, ClusterIP, NodePort, kube-proxy, CoreDNS, Endpoints

## Objective

Expose a Deployment via ClusterIP and NodePort, prove service discovery works through CoreDNS, and debug a deliberately-broken selector.

## Concepts Covered

• How Endpoints objects bind Services to Pods

• ClusterIP vs NodePort vs LoadBalancer

• DNS records created per Service

## Prerequisites

• Lab 4 deployment `web` running

## Architecture

A Service is a virtual IP. kube-proxy programs DNAT rules on every node so requests to ClusterIP land on a real Pod IP.

```
Client
   |
   v
Service (ClusterIP 10.96.x.x)
   |
   v   (kube-proxy iptables/IPVS rules)
   v
Endpoints  -->  Pod  Pod  Pod  (by label selector)
```

## Lab Steps

### *01. Create a ClusterIP service*

Creates a stable virtual IP that load-balances across all Pods with matching labels.

```
kubectl expose deploy/web --port=80 --target-port=80 --name=web-svc
kubectl get svc web-svc
```

**Expected:** ClusterIP assigned from 10.96.0.0/12 range.

**Internals:** Endpoints controller writes the Pod IPs into an Endpoints object with the same name as the Service.

### *02. Inspect Endpoints*

Endpoints == the actual targets behind the virtual IP.

```
kubectl get endpoints web-svc
```

**Expected:** Comma-separated list of PodIP:80.

**Internals:** If this list is empty your selector is wrong or no Pod is ready.

### *03. Service discovery via DNS*

DNS resolves the service name to its ClusterIP; HTTP returns the nginx welcome page.

```
kubectl run probe --rm -it --image=busybox -- sh -c 'nslookup web-svc.default.svc.cluster.local; wget -qO- we
```

**Expected:** Address: 10.96.x.x, then `<!DOCTYPE html>`.

**Internals:** CoreDNS holds a zone for cluster.local and answers from the in-memory Service/Endpoints cache.

### *04. Promote to NodePort*

Allocates a port in 30000-32767 on every node forwarding to the Service.

```
kubectl patch svc web-svc -p '{"spec":{"type":"NodePort"}}'
kubectl get svc web-svc
```

**Expected:** PORT(S) shows `80:3xxxx/TCP`.

**Internals:** kube-proxy adds an extra iptables rule chain for the node port.

### *05. Break the selector on purpose*

Empty Endpoints == 503 / Connection refused from clients. Classic outage.

```
kubectl patch svc web-svc -p '{"spec":{"selector":{"app":"wrong"}}}'
kubectl get endpoints web-svc
```

**Expected:** Endpoints list becomes <none>.

**Internals:** Endpoints controller can find no Pod matching the new selector.

### *06. Fix it*

Restoring the right selector immediately repopulates Endpoints.

```
kubectl patch svc web-svc -p '{"spec":{"selector":{"app":"web"}}}'
```

**Expected:** Endpoints list filled again.

**Internals:** No Pod restart needed — only the Service object changed.

## Common Errors

**503 from a healthy Pod** — Cause: Selector mismatch → empty Endpoints · Fix: kubectl get endpoints <svc>

**DNS lookup times out** — Cause: CoreDNS down or NetworkPolicy blocking egress to kube-dns · Fix: Check coredns pods and any default-deny NetworkPolicy

## Troubleshooting

• Endpoints empty → selector or readiness

• Endpoints full but no connectivity → kube-proxy or CNI

• DNS slow → CoreDNS HPA + ndots:5

## Enterprise Relevance

Service mesh, Ingress and LoadBalancer all sit on top of these primitives. Get them wrong and everything downstream is unreliable.

## Validation

• You can reach a Pod by service DNS name

• You can detect a selector mismatch in <30s using `kubectl get endpoints`

## Related Incidents & Guide Sections

Incidents: K8S-INC-3003

• Networking & Service types

• Service unreachable

## Questions

### *Beginner*

**Q.** What's the DNS name of a Service called `web-svc` in namespace `default`?

**A.** web-svc.default.svc.cluster.local

### *Intermediate*

**Q.** What populates the Endpoints object?

**A.** The endpoints controller, based on the Service selector + Pod readiness.

### *Advanced*

**Q.** Why use IPVS over iptables for kube-proxy?

**A.** O(1) rule lookup at large scale vs O(n) for iptables.

### *Scenario*

**Q.** All clients report 503 but Pods are Running. First check?

**A.** `kubectl get endpoints <svc>` — if empty, look at selector/readiness.

### *Viva*

**Q.** Difference between Service and Ingress?

**A.** Service = L4 in-cluster VIP. Ingress = L7 HTTP router, usually exposed via a single LoadBalancer.